ĐẠI HỌC BÁCH KHOA HÀ NỘI

Trường Công nghệ Thông tin và Truyền thông



Efficient RAG

Nén ngữ cảnh và truy hồi hiệu quả cho hỏi đáp với LLM

Đề tài 13: Efficient LLM/RAG – Token Compression, Context Compression và Efficient Retrieval

Sinh viên Nguyễn Hoàng Sơn (20261127M)
Võ Việt Hoàng (20251064M)
Giảng viên Lê Thanh Hương
Học phần IT4772 – Xử lý ngôn ngữ tự nhiên

Nội dung

I. Bài toán

- Chi phí ngữ cảnh dài trong RAG
- Hiện tượng *lost in the middle*

II. Phương pháp

- Benchmark & giao thức thí nghiệm
- 4 nhóm nén cứng · truy hồi hiệu quả
- **Nén mềm “encode-once”** (đề xuất)

III. Kết quả

- Giữ bằng chứng (không cần LLM)
- Độ chính xác – token – vị trí
- So với số liệu công bố
- Nén mềm · truy hồi · CPU vs GPU

IV. Tổng kết

- Demo · hạn chế · kết luận

Bài toán: ngữ cảnh dài vừa đắt vừa khó dùng

- RAG đưa *k* đoạn văn vào prompt: 20 đoạn NQ $\approx$ **2.900 token** cho một câu hỏi vài chục token.
- Chi phí & độ trễ prefill tăng tuyến tính (attention: bậc hai) theo độ dài.
- **Lost in the middle** (Liu et al., 2024): LLM dùng kém thông tin nằm giữa ngữ cảnh – đường cong chữ U theo vị trí đoạn chứa đáp án.
- Hai hướng giảm chi phí:
  - **Nén ngữ cảnh** (Track A): bỏ bớt token/câu/đoạn thừa.
  - **Truy hồi hiệu quả** (Track B): lấy đúng đoạn với chỉ mục nhỏ, nhanh.

Câu hỏi nghiên cứu

RQ1. Nén 2–4$\times$ mất bao nhiêu độ chính xác? Có làm phẳng đường cong vị trí?

RQ2. Truy hồi hiệu quả: đánh đổi chất lượng – tốc độ – bộ nhớ?

RQ3. Kết luận của bài báo (GPT-3.5) còn đúng với LLM mở 7B?

RQ4. Không có GPU thì có làm được không?

RQ5. Nén mềm "mã hoá một lần" (vector) có thay được văn bản?

Hệ thống: truy hồi → nén → sinh câu trả lời



Mã nguồn gói Python `effrag` (uv, Python 3.11): mỗi bộ nén kế thừa `Compressor`, mỗi retriever có `search(queries, k)`; kết quả ghi chung một schema (`summary.csv` + `.jsonl` từng câu).

Sinh câu trả lời Qwen2.5-7B-Instruct-AWQ trên vLLM (GPU 12 GB), `temperature=0`, prompt gốc của Lost in the Middle; cache theo hash prompt ⇒ chạy lại không tốn chi phí.

Benchmark & giao thức thí nghiệm

Dữ liệu

- NQ-Open 20 đoạn của *Lost in the Middle* (2.655 câu); chọn **500 câu** cố định (seed 42).
- Đoạn vàng ở vị trí **1, 10, 20** – đúng 3 điểm có số công bố của LongLLMLingua / LLMLingua-2.

Chỉ số

- Accuracy = đáp án chuẩn hoá nằm trong output (best-subspan EM).
- **Recall bằng chứng**: đáp án còn trong ngữ cảnh sau nén (không cần LLM).
- Token (tiktoken `o200k_base`), tỉ lệ nén, độ trễ nén.

Thiết kế tiết kiệm tài nguyên

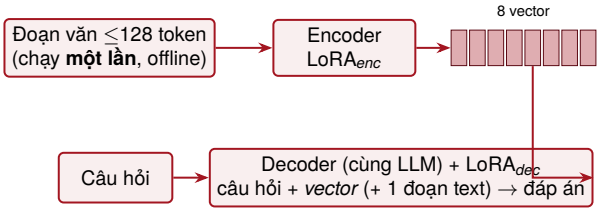
- **Dry-run** mọi bộ nén trước (miễn phí) → chỉ gọi LLM cho cấu hình đáng chạy.
- Bộ nén theo *từng đoạn* được **memo hoá** qua các vị trí.
- Dùng **số liệu đã công bố** làm tham chiếu thay vì chạy lại.
- Tham chiếu bất biến theo vị trí (closed-book, oracle) chạy 1 lần.
- Một job GPU nặng tại một thời điểm; mọi model chạy được trên CPU.

Các phương pháp nén cứng được so sánh

Mức	Phương pháp	Ý tưởng	Nguồn
Prompt	Truncate	cắt đầu ngữ cảnh theo ngân sách token	baseline
Tài liệu	BM25 / Dense / Rerank top-k	chấm điểm đoạn theo câu hỏi, giữ k đoạn, *giữ thứ tự gốc*	bm25s, bge-small, MiniLM CE
Token	LLMLingua-2	phân loại giữ/bỏ từng token (XLM-R, chưng cất từ GPT-4)	arXiv 2403.12968
	LongLLMLingua	perplexity có điều kiện câu hỏi, ngân sách theo đoạn	arXiv 2310.06839
	Rerank-10 $\rightarrow$ LLMLingua-2	lọc đoạn rồi nén token	LLMLingua-2 Tab. 4
Câu	Provence	cắt tỉa câu theo câu hỏi, gộp với reranker (DeBERTa)	arXiv 2501.16214
	OpenProvence	bản mở, ModernBERT, đa ngôn ngữ	hotchpotch (2025)
Vector	PISCO	mỗi đoạn $\rightarrow$ 8 vector bộ nhớ (16$\times$), Mistral-7B	arXiv 2501.16075
	Nén mềm encode-once	**đề xuất của nhóm** (slide sau)	–

Ý chính. *Tỉ lệ mục tiêu 2$\times$ ($\approx$1.450 token) và 4$\times$ ($\approx$730 token) như bảng của LongLLMLingua; phương pháp dùng ngưỡng (Provence) báo cáo tỉ lệ thực tế.*

Đề xuất: nén mềm “encode-once”



- Qwen2.5-1.5B-Instruct, 2 adapter LoRA (r=16) trên *một* model.
- 15k câu Tevatron NQ-train (loại câu trùng test), vị trí vàng ngẫu nhiên, $k \in \{5, 10, 20\}$.
- **Biến thể lai**: đoạn top-1 của reranker giữ dạng text (tương tự *lightning indexer* của DeepSeek-V3.2: một bộ chấm nhẹ chọn phần được xem đầy đủ).

Vì sao?

Nén cứng chạy lại *mỗi câu hỏi*. Nén mềm mã hoá kho tài liệu **một lần**; lúc hỏi chỉ đưa 8 vector/đoạn (20 đoạn: 160 vị trí thay vì ≈2.900 token).

Cổng dừng (21/10)

200 câu dev: tiếp tục nếu acc ≥ closed-book + 10 điểm và ≥ 70% full-text cùng reader; nếu không ⇒ báo cáo phân tích thất bại.

Đối chứng: PISCO (Naver) huấn luyện trên >450k mẫu, 7B; nhóm thử ở quy mô BTL (15k mẫu, 1.5B, 1 GPU 12 GB).

Kết quả 1: bộ nén giữ được bao nhiêu bằng chứng? (không cần LLM)

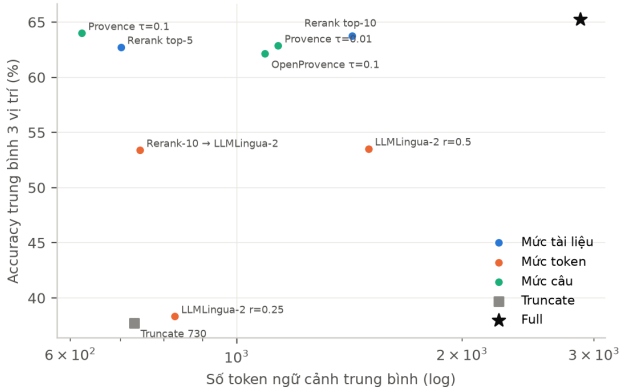
Bộ nén	Token	Nén	Recall
Truncate 730	730	3,9×	100 / 16 / 16†
BM25 top-5	705	4,1×	53,2
Dense top-5	702	4,1×	69,0
Rerank top-5	701	4,1×	79,4
LLMLingua-2 r=0,25	827	3,5×	68,0
Rerank-10 → LLMLingua-2	743	3,9×	79,2
LongLLMLingua 730t‡	674	4,3×	77,5
Provence τ=0,1	621	**4,6**×	**91,6**
Rerank top-10	1.425	2,0×	89,8
LLMLingua-2 r=0,5	1.500	1,9×	88,2
OpenProvence τ=0,1	1.091	2,6×	**96,4**

† Truncate: recall ở vị trí 1 / 10 / 20 (cắt mất các đoạn phía sau). Các bộ nén khác không phụ thuộc vị trí. ‡ n=200, vị trí 10.

- Cùng ≈4×, **Provence giữ 91,6%** bằng chứng, hơn rerank top-5 12 điểm, hơn LLMLingua-2 24 điểm.
- Nén mức *câu* theo câu hỏi giữ ngữ nghĩa tốt hơn bỏ token rời rạc.
- BM25 trong 20 đoạn đã là hard negative ⇒ chỉ 53%.

Ý chính. *Recall là trần trên của accuracy sau nén, đo được miễn phí trên CPU.*

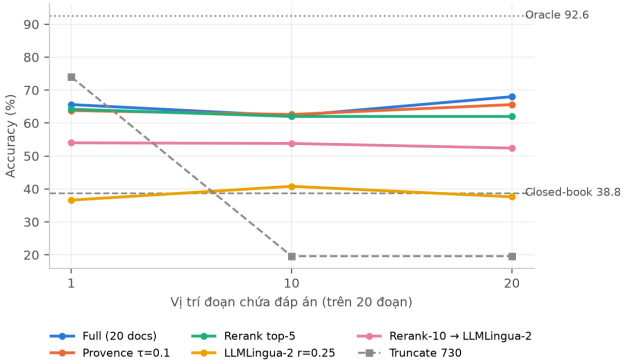
Kết quả 2: độ chính xác – số token (Qwen2.5-7B, 500 câu)



- Full 20 đoạn: **65,3**% (TB 3 vị trí); closed-book 38,8; oracle 92,6.
- **Provence τ=0,1**: 621 token (4,6×), **64,0**% – không giảm có ý nghĩa ở vị trí nào (bootstrap 95%).
- Rerank top-5 (4,1×): 62,7%; giảm $-6{,}0$ điểm ở vị trí 20.
- **LLMLingua-2** *r*=0,25: 38,3% ≈ closed-book, dù recall còn 68%.

Ý chính. *Nén mức câu theo câu hỏi: 4,6× ít token, gần như không mất độ chính xác. Nén mức token làm hỏng văn bản đối với reader 7B.*

Kết quả 3: nén và hiện tượng *lost in the middle*



- Full: 65,6 / 62,2 / 68,0 – gần như **phẳng** (chênh 5,8 điểm) so với chữ U của GPT-3.5: 75,7 / 54,1 / 63,1 (chênh 21,6).
- Provence: 63,8 / 62,6 / 65,6 – song song với Full: không còn nhiều để "làm phẳng".
- Truncate: 74,0 ở vị trí 1 (**+8,4**, bớt nhiễu) nhưng 19,6 ở vị trí 10/20.

Ý chính. *Reader hiện đại (ngữ cảnh 32K+) hầu như không còn "lost in the middle" ở 3K token.*

Kết quả 4: so với số liệu đã công bố (GPT-3.5-Turbo)

GPT-3.5-Turbo (công bố)	@1	@10	@20	Qwen2.5-7B-AWQ (nhóm)	@1	@10	@20
Original prompt	75.7	54.1	63.1	Full (20 docs)	65.6	62.2	68.0
BM25 (4×)	-35.1	-15.9	-26.5	BM25 top-5	–	-16.8	–
SBERT (4×)	-8.8	+4.9	-2.8	Dense top-5	–	-6.2	–
LLMLingua-2 (4×)	-27.1	-10.5	-23.2	LLMLingua-2 r=0.25	-29.0	-21.4	-30.4
LLMLingua-2 + lọc tài liệu (4×)	-1.7	+12.9	+2.2	Rerank-10 → LLMLingua-2	-11.6	-8.4	-15.6
LongLLMLingua (4×)	-0.7	+17.1	+11.6	LongLLMLingua 730t	–	+1.0	–

Hàng đầu: accuracy tuyệt đối của ngữ cảnh đầy đủ; các hàng sau: chênh lệch (điểm) so với ngữ cảnh đầy đủ của chính hệ đó. LongLLMLingua: so cặp 193 câu.

Tái lập được: BM25 $\ll$ Dense; LLMLingua-2 đơn lẻ ở 4× sụt nặng (-21 đến -30 điểm, công bố -11 đến -27).

Không tái lập: "nén làm *tăng* accuracy ở giữa" (LongLLMLingua $+17{,}1 \rightarrow +1{,}0$ n.s.) – vì reader mới không còn lost in the middle.

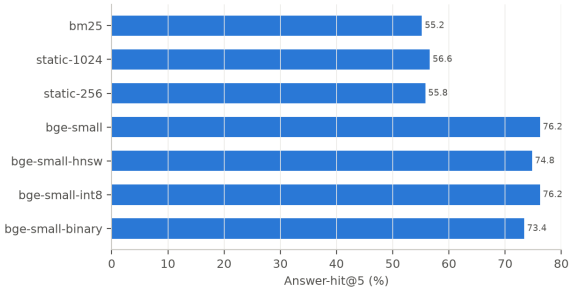
Kết quả 5: nén mềm (PISCO và mô hình của nhóm)

Cấu hình	@1	@10	@20	TB	Token	Nén
Mistral-7B-Instruct-v0.2 (nf4)						
Văn bản (20 đoạn)	56.6	63.2	61.2	60.3	2875	1.0×
Văn bản (rerank top-5)	–	63.0	–	–	701	4.1×
Oracle	89.6	89.6	89.6	89.6	113	25.5×
Closed-book	36.2	36.2	36.2	36.2	0	–
PISCO 16× (20 đoạn)	47.8	52.8	44.0	48.2	160	18.0×
PISCO 16× (rerank top-5)	–	56.2	–	–	40	71.9×

Mỗi khối: cùng một reader, chỉ khác ngữ cảnh văn bản / vector. Rerank: chỉ vị trí 10.

- **PISCO**: 160 vector thay 2.875 token (18×), TB 48,2% so với 60,3% văn bản cùng reader.
- Giảm 9 / 10 / 17 điểm ở vị trí 1 / 10 / 20, đều có ý nghĩa; tệ nhất khi đoạn vàng ở cuối.
- Rerank-5 → PISCO: 56,2% với 40 vector (72×), vẫn kém văn bản rerank-5 6,8 điểm.
- **[Mô hình của nhóm: mềm / lai]**

Kết quả 6: truy hồi hiệu quả (corpus gộp 50,7k đoạn, 500 câu)



Hệ thống	hit@5	Chỉ mục
BM25S	55,2	–
Static embedding 256d	55,8	49,5 MB
bge-small fp32	76,2	74,3 MB
bge-small **int8**	**76,2**	**18,6 MB**
binary + rescore	73,4	**2,3 MB**

Ý chính. *Lượng tử hoá int8 giảm 4× bộ nhớ không mất điểm; binary + rescore giảm 32× chỉ mất 2,8 điểm.*

Kết quả 7: không có GPU thì sao?

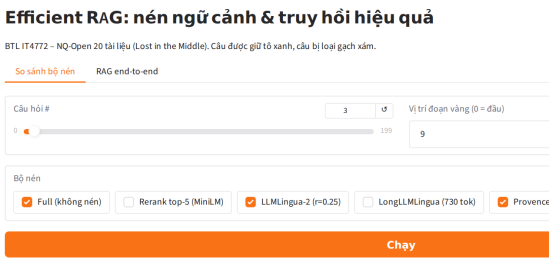
Bộ nén	GPU	CPU	×
BM25 top-5	2	2	0.9×
Rerank top-5	16	196	12.4×
Dense top-5	26	361	13.7×
OpenProvence τ=0.1	77	1598	20.8×
Provence τ=0.1	200	4906	24.6×
LLMLingua-2 r=0.5	455	13104	28.8×
LongLLMLingua 730t	625	14837	23.8×

Trung vị ms/câu, 20 câu, không tính LLM. RTX 4070 SUPER vs i5-14600K 8 luồng.

- **Mức tài liệu** $<$ 0,4 s trên CPU: dùng tương tác được.
- **Mức câu** chậm 21–25×: Provence 4,9 s, OpenProvence 1,6 s.
- **Mức token** 13–15 s và accuracy kém hơn $\Rightarrow$ không nên dùng khi thiếu GPU.
- Reader qua API: Provence giảm **78%** token đầu vào, $-1{,}3$ điểm (n.s.).

Ý chính. *Không có GPU vẫn chạy được: rerank top-5 (0,2 s) hoặc Provence (4,9 s) trên CPU + reader qua API.*

Demo (Gradio): so sánh bộ nén trên cùng một câu hỏi





- Token trước/sau, thời gian nén, đáp án còn trong ngữ cảnh, câu trả lời LLM.
- Câu được giữ tô xanh, câu bị loại gạch ngang; đoạn vàng được đánh dấu.
- Tab 2: câu hỏi tự do → BM25S/bge-small (50,7k đoạn) → nén → LLM.
- Chạy hoàn toàn trên CPU; không có LLM vẫn xem được phần nén.

Hạn chế & kết luận

Hạn chế

- Một LLM 7B lượng tử hoá, một tập (NQ); số công bố dùng GPT-3.5 đã ngừng.
- 500 câu, 3 vị trí; LongLLMLingua chỉ 200 câu ở vị trí 10.
- Nén mềm huấn luyện ở quy mô nhỏ (15k mẫu, 1.5B).

Kết luận

- **Chọn câu > chọn tài liệu > xoá token**: Provence 4,6×, −1,3 điểm (n.s.); LLMLingua-2 4× về mức closed-book.
- Reader mới gần như hết *lost in the middle* ⇒ lợi ích "nén làm tăng accuracy" của GPT-3.5 không còn.
- Không GPU vẫn làm được: nén trên CPU + reader qua API; dense nhỏ int8 cho truy hồi.

Cảm ơn thầy cô và các bạn đã lắng nghe!